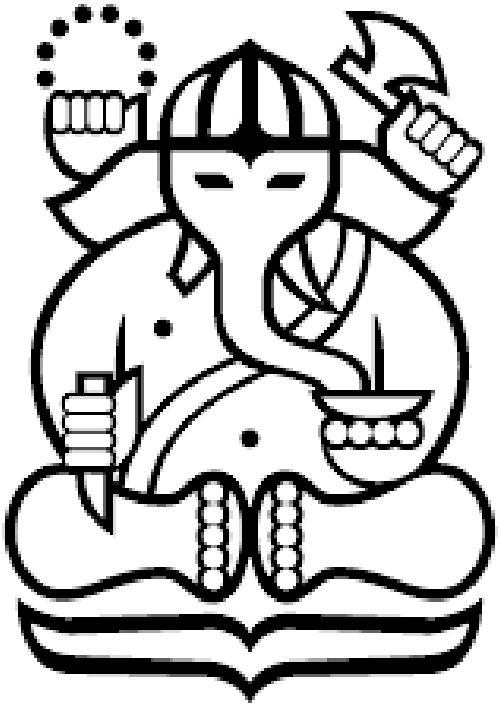
Laporan Tugas Kecil 2

IF2211 Strategi Algoritma

Voxelization Objek 3D Menggunakan Struktur Data Octree



Disusun Oleh :

Timothy Bernard Soeharto - 13524092

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Semester II 2025/2026

A. Struktur Data yang Digunakan

1. Mesh

Mesh menyimpan dua komponen utama, yaitu daftar vertex dan daftar face. Vertex direpresentasikan sebagai titik 3D (x, y, z), sedangkan face disimpan sebagai tiga indeks vertex yang membentuk sebuah segitiga. Selama proses pembacaan file OBJ, program juga menghitung bounding box mesh, yaitu nilai minimum dan maksimum pada sumbu x, y, dan z.

2. Octree

Octree adalah struktur data pohon spasial untuk ruang 3D, di mana setiap node merepresentasikan sebuah kubus. Sebuah node internal dapat dibagi menjadi delapan anak. Dalam implementasi ini, setiap OctreeNode menyimpan informasi berupa pusat kubus (cx, cy, cz), halfSize, depth, state, serta array children[8]. Nilai state digunakan untuk menandai apakah node tersebut kosong (**EMPTY**) atau merupakan calon boundary voxel (**BOUNDARY**).

3. Root Cube

Root cube dibentuk dari bounding box mesh. Titik pusatnya diambil dari titik tengah bounding box, sedangkan panjang sisi kubus ditentukan berdasarkan extent terbesar dari ketiga sumbu. Nilai tersebut kemudian dibagi dua untuk memperoleh **halfSize**. Selain itu, ditambahkan sedikit padding agar seluruh mesh benar-benar berada di dalam root cube dan untuk mengurangi potensi masalah numerik ketika objek berada tepat pada batas kubus.

B. Algoritma Divide and Conquer yang Digunakan

Inti dari penyelesaian masalah ini adalah membagi ruang 3D secara bertahap, bukan langsung memeriksa seluruh voxel kecil sejak awal. Pendekatan ini termasuk divide and conquer karena masalah besar, yaitu menentukan voxel mana yang beririsan dengan boundary mesh dipecah menjadi submasalah yang sama pada delapan subkubus yang lebih kecil.

1. Langkah-Langkah Umum Algoritma

Rangkuman langkah-langkah algoritma:

1. Mulai dari sebuah root cube yang membungkus seluruh objek.
2. Periksa apakah node octree beririsan dengan salah satu segitiga pada mesh.
3. Jika node tidak beririsan dengan boundary mesh, proses pada node tersebut dihentikan.
4. Jika node beririsan dan kedalaman maksimum belum tercapai, node dibagi menjadi 8 subkubus.
5. Proses yang sama diterapkan kembali pada setiap anak yang masih berstatus boundary.
6. Ketika depth maksimum tercapai, leaf node boundary dipertahankan sebagai voxel keluaran.

2. Langkah-Langkah Algoritma Secara Rinci

Program dimulai dengan membaca file OBJ baris per baris. Saat menemukan baris vertex, posisi titik disimpan ke dalam vektor vertex. Saat menemukan baris face, tiga indeks vertex disimpan ke dalam vektor face. Selama proses ini, bounding box global mesh juga terus diperbarui.

Setelah itu, program membentuk root cube berdasarkan bounding box mesh. Dari bounding box tersebut dihitung titik pusat dan dipilih perbedaan min dan max terbesar dari tiga sumbu yang ada, yang kemudian digunakan untuk membangun node awal octree.

Pada setiap node, program memeriksa apakah kubus node mungkin beririsan dengan segitiga mesh. Tahap pertama adalah AABB check, yaitu membandingkan bounding box segitiga dengan bounding box kubus node. Jika lolos, program melanjutkan ke tahap kedua, yaitu triangle-box intersection check berbasis Separating Axis Theorem (SAT) untuk memastikan apakah overlap benar-benar terjadi.

Jika sebuah node beririsan dengan boundary mesh dan kedalamannya masih di bawah batas maksimum, node tersebut dibagi menjadi delapan anak. Masing-masing anak memiliki halfSize (**childHalfSize**) sebesar setengah dari parent, dan pusatnya digeser sejauh **±childHalfSize** pada sumbu x, y, dan z.

Setelah anak-anak terbentuk, setiap anak diperiksa kembali. Anak yang berstatus **EMPTY** tidak diproses lebih lanjut, sedangkan anak yang berstatus **BOUNDARY** akan dibagi lagi dengan prosedur yang sama. Rekursi berhenti ketika node kosong atau ketika kedalaman maksimum telah tercapai. Dengan algoritma ini, pembagian ruang hanya dilakukan di area/node yang dekat dengan permukaan objek.

Pada tahap akhir, semua leaf node pada depth maksimum yang berstatus **BOUNDARY** ditulis ke file OBJ sebagai kubus. Setiap kubus terdiri atas 8 vertex dan 12 face segitiga.

3. Pseudocode Algoritma Divide and Conquer

```
PROCEDURE VoxelSubdivider(node, maxDepth, mesh)

   IF node.state == EMPTY THEN
      RETURN
   END IF

   IF node.depth == maxDepth THEN
      MARK node AS FINAL VOXEL
      RETURN
   END IF

   childHalfSize <- node.halfSize / 2
   childIndex <- 0
```

```
  FOR EACH sx IN {-1, +1}
    FOR EACH sy IN {-1, +1}
      FOR EACH sz IN {-1, +1}

        childCenterX <- node.cx + sx * childHalfSize
        childCenterY <- node.cy + sy * childHalfSize
        childCenterZ <- node.cz + sz * childHalfSize

        childState <- OverlapChecker(
          mesh,
          childCenterX,
          childCenterY,
          childCenterZ,
          childHalfSize
        )

        IF childState != EMPTY THEN
          child <- new OctreeNode(
            childCenterX,
            childCenterY,
            childCenterZ,
            childHalfSize,
            node.depth + 1
          )

          child.state <- childState
          node.children[childIndex] <- child

          VoxelSubdivider(child, maxDepth, mesh)
        ELSE
          node.children[childIndex] <- NULL
        END IF

        childIndex <- childIndex + 1

      END FOR
    END FOR
  END FOR

END PROCEDURE
```

Pseudocode di atas menunjukkan inti algoritma *divide and conquer* pada proses voxelisasi dengan struktur data **octree**. Proses dimulai dari satu node yang merepresentasikan sebuah kubus dalam ruang 3D. Jika node sudah mencapai kedalaman maksimum atau berstatus **EMPTY**, maka proses rekursi dihentikan karena kubus tersebut tidak perlu dibagi lagi.

Jika node masih perlu diproses, kubus akan dibagi menjadi delapan sub-kubus yang lebih kecil. Setelah itu, setiap ***child cube*** diperiksa menggunakan **OverlapChecker** untuk menentukan apakah kubus tersebut masih beririsan dengan boundary mesh. Jika sebuah child berstatus **EMPTY**, maka child tersebut diabaikan dan tidak diproses lebih lanjut. Sebaliknya, jika child masih beririsan dengan boundary mesh, maka fungsi **VoxelSubdivider** dipanggil kembali secara rekursif pada child tersebut.

Dengan mekanisme ini, pembagian ruang hanya dilakukan pada bagian yang relevan terhadap permukaan objek. Pendekatan ini disebut *divide and conquer* karena satu masalah besar, yaitu voxelisasi pada sebuah kubus, dipecah menjadi delapan submasalah yang lebih kecil. Masing-masing submasalah kemudian diselesaikan dengan cara yang sama secara rekursif hingga mencapai kondisi berhenti. Strategi ini membuat proses menjadi lebih efisien dibandingkan pemeriksaan seragam pada seluruh ruang, karena area kosong dapat dieliminasi lebih awal.

C. Pemeriksaan Overlap

1. AABB Check

AABB check digunakan sebagai broad-phase test. Untuk setiap segitiga, dihitung nilai minimum dan maksimum koordinat x, y, dan z dari ketiga vertex. Nilai-nilai tersebut membentuk axis-aligned bounding box segitiga. Node octree juga memiliki bounding box yang diperoleh dari pusat kubus dan halfSize. Jika kedua bounding box tidak saling overlap pada salah satu sumbu, maka segitiga tersebut pasti tidak beririsan dengan node, sehingga pemeriksaan yang lebih mahal tidak perlu dilakukan.

2. Triangle-Box Intersection Check

Jika sebuah segitiga lolos AABB check, program melanjutkan ke pemeriksaan overlap yang lebih akurat menggunakan Separating Axis Theorem (SAT). Secara umum, dua objek yang tidak konveks (kubus dan segitiga tidak konveks) tidak saling overlap jika terdapat sebuah sumbu pemisah yang membuat proyeksi kedua objek pada sumbu tersebut tidak beririsan.

Dalam kasus segitiga terhadap kubus axis-aligned, sumbu yang diuji meliputi tiga sumbu kubus (x, y, z), satu normal segitiga, dan sembilan sumbu hasil cross product antara tiga edge segitiga dengan tiga sumbu kubus. Jika ditemukan satu saja sumbu dengan interval proyeksi yang terpisah, maka segitiga dan kubus tidak overlap. Sebaliknya, jika tidak ada sumbu pemisah, berarti keduanya benar-benar beririsan.

D. Kompleksitas dan Analisis Singkat

Pendekatan brute force harus memeriksa semua voxel kecil terhadap semua segitiga pada mesh. Jika jumlah voxel akhir adalah **V** dan jumlah segitiga pada mesh adalah **T**, maka kompleksitas brute force berada pada orde **O(V*T)**. Pada depth atau resolusi tinggi, jumlah voxel dapat menjadi sangat besar, sehingga pendekatan ini menjadi mahal baik dari sisi waktu maupun memori.

Pada program ini digunakan struktur data **octree** dengan pendekatan **divide and conquer**. Setiap node hanya akan dibagi lebih lanjut apabila masih beririsan dengan boundary mesh. Node yang berstatus EMPTY akan langsung dihentikan dan tidak diproses lagi. Dengan demikian, ruang kosong yang besar dapat dieliminasi lebih awal tanpa perlu diperiksa sampai ke voxel terkecil.

Jika maksimum depth adalah d, maka dalam kasus terburuk jumlah node octree dapat mencapai:

$$1+8+8^2+\cdots+8^d=O(8^d)$$

Pada setiap node, fungsi overlap checker masih melakukan iterasi terhadap seluruh face mesh. Jika jumlah segitiga adalah **T**, maka biaya pemeriksaan satu node adalah **O(T)**. Oleh karena itu, kompleksitas waktu dalam kasus terburuk menjadi:

$$O(8^d \cdot T)$$

Kasus terburuk ini terjadi apabila hampir semua node tetap beririsan dengan mesh sehingga rekursi terus berlangsung sampai depth maksimum.

Namun, pada praktiknya kompleksitas nyata biasanya lebih kecil dari itu, karena tidak semua node akan dibagi sampai kedalaman maksimum. Banyak node kosong dapat dieliminasi lebih awal. Oleh sebab itu, penggunaan octree memberikan efisiensi yang jauh lebih baik dibanding brute force penuh, terutama pada objek yang boundary-nya hanya menempati sebagian kecil dari ruang bounding cube.

Dari sisi memori, kompleksitas ruang pada kasus terburuk juga mengikuti jumlah node octree, yaitu sebesar:

$$O(8^d)$$

karena setiap node dapat memiliki hingga 8 child. Akan tetapi, seperti pada kompleksitas waktu, penggunaan memori aktual biasanya lebih kecil karena banyak cabang rekursi berhenti lebih awal pada node kosong.

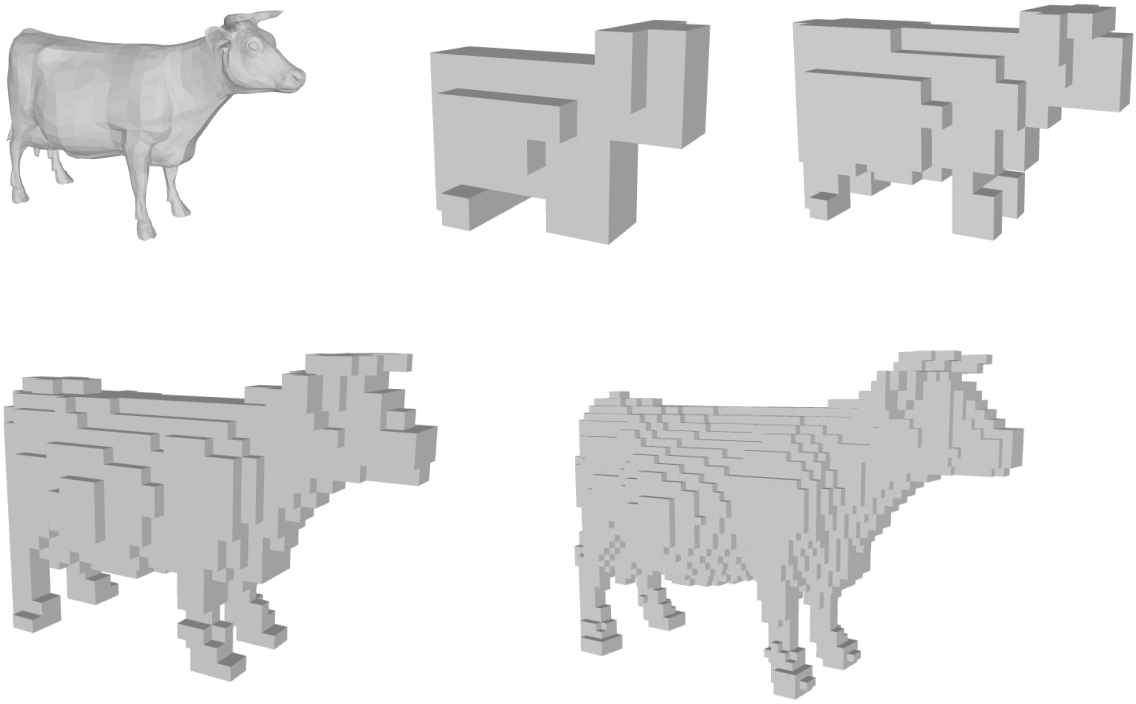
E. Deskripsi Implementasi Program

Komponen-komponen utama program:

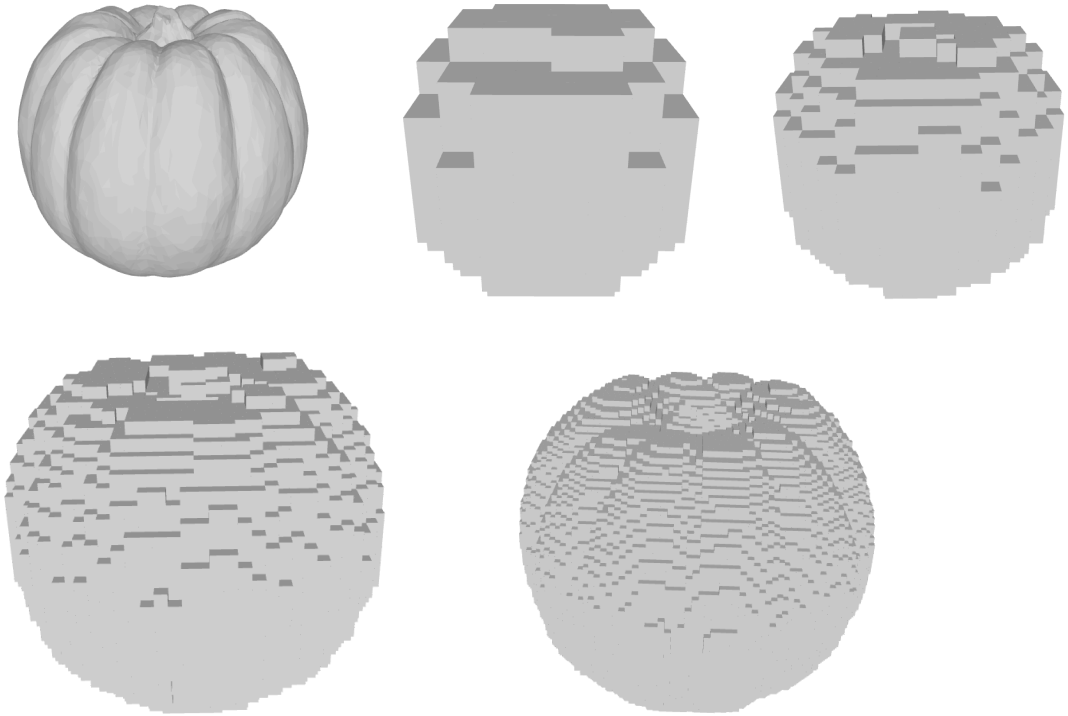
- objLoader: membaca file OBJ, menyimpan vertex dan face, menghitung bounding box, serta menghasilkan informasi root cube. Hanya menerima faces dengan tiga vertices saja.
- overlapChecker: menentukan state node dengan menggabungkan AABB check dan triangle-box intersection check.
- voxelSubdivider: membagi node menjadi delapan anak secara rekursif hingga kedalaman maksimum, dengan hanya melanjutkan proses pada node yang tidak kosong.
- objOutput: menelusuri octree untuk mencari leaf node boundary pada depth maksimum, lalu menuliskannya ke file OBJ sebagai kubus dengan 12 face segitiga.
- main: menyediakan antarmuka CLI untuk membaca nama file masukan, depth maksimum, dan nama file keluaran, lalu menjalankan seluruh subprogram dengan urutan yang benar.

F. Hasil Pengujian

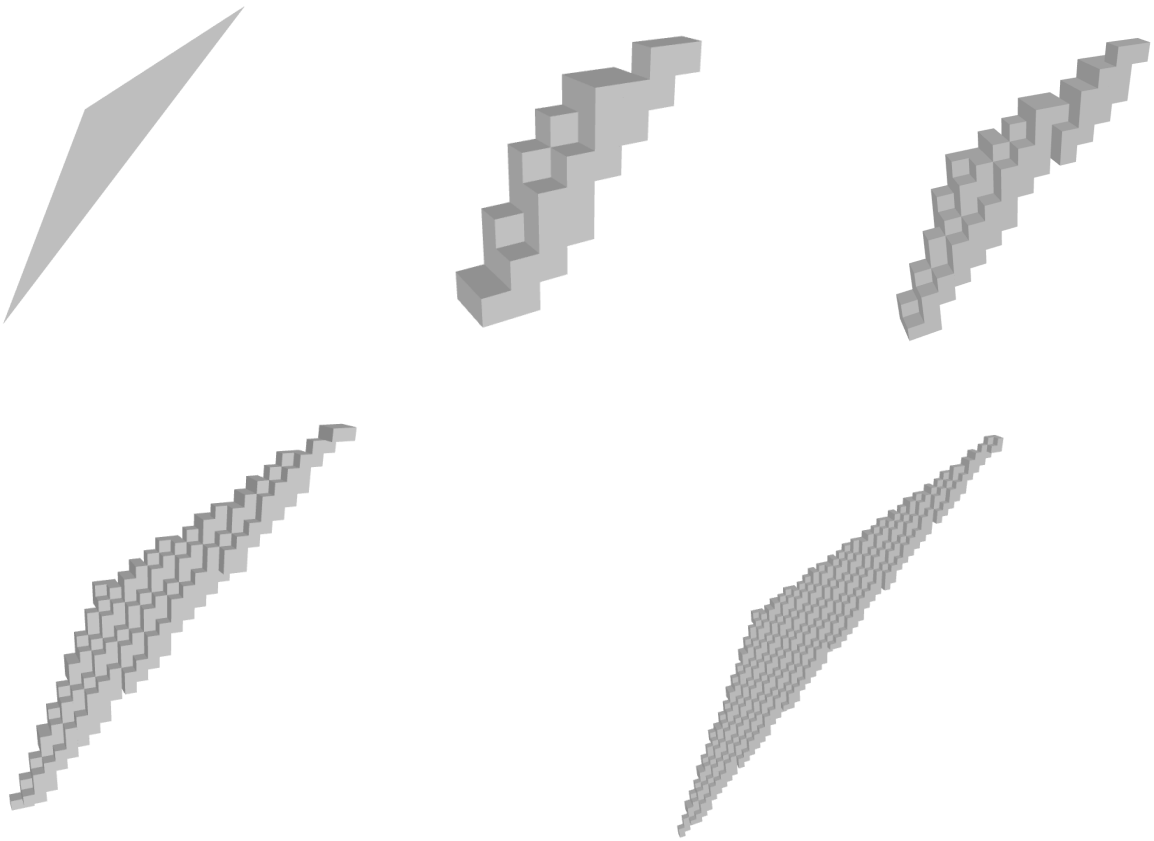
1. **Cow (depth 3-6):**



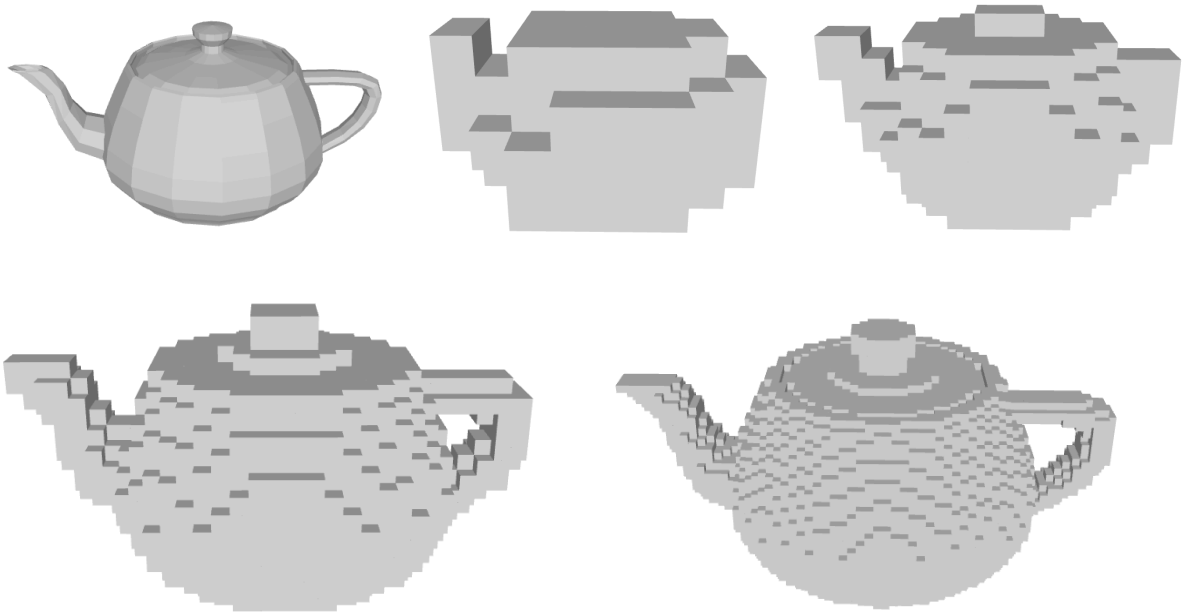
2. **Pumpkin (depth 3-6):**



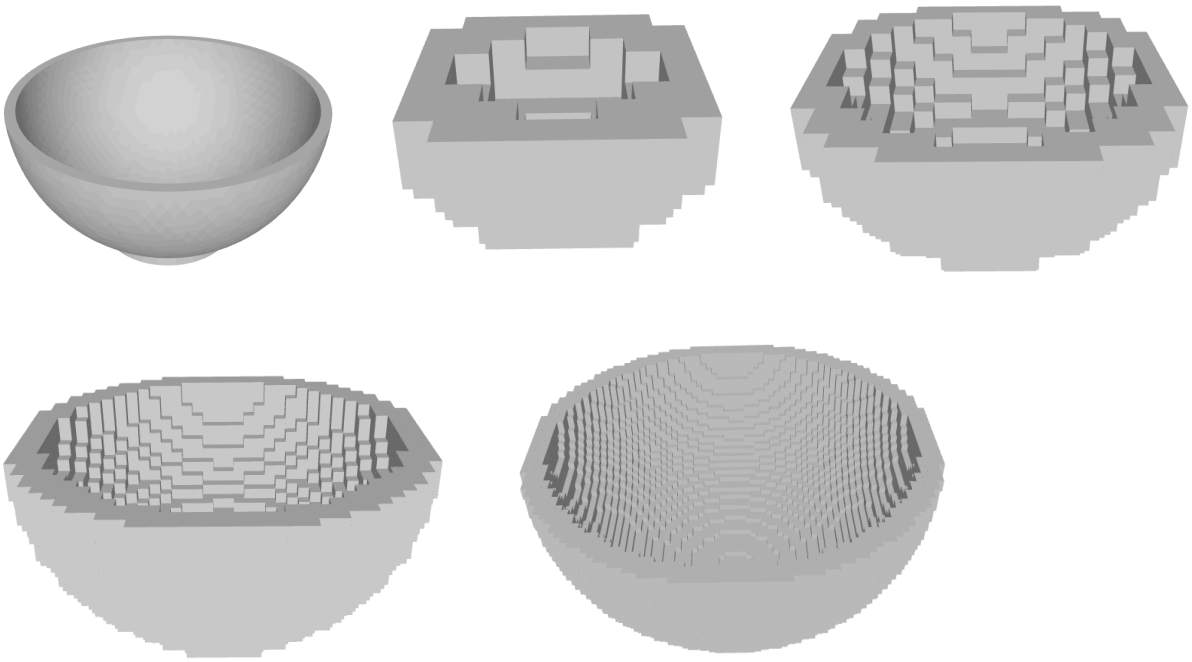
3. **Line (depth 3-6):**



4. **Teapot (depth 3-6):**



5. **Bowl (depth 3-6):**



G. Analisis Performa dari Pengujian

1. **Tabel Informasi Report dari Pengujian Generasi Objek Voxel**

No	Model	Kedalaman	Waktu Eksekusi (ms)	Jumlah Voxel
1	`cow.obj`	3	51	88
2	`cow.obj`	4	209	332
3	`cow.obj`	5	832	1362
4	`cow.obj`	6	3567	5621
5	`teapot.obj`	3	14	108
6	`teapot.obj`	4	30	418
7	`teapot.obj`	5	126	1748
8	`teapot.obj`	6	505	7303
9	`pumpkin.obj`	3	184	277
10	`pumpkin.obj`	4	1106	1123

11	pumpkin.obj	5	5135	4626
12	pumpkin.obj	6	22827	18635
13	line.obj	3	0.05	32
14	line.obj	4	0.36	91
15	line.obj	5	0.41	284
16	line.obj	6	1.35	1006
17	bowl.obj	3	131	160
18	bowl.obj	4	823	784
19	bowl.obj	5	4433	4292
20	bowl.obj	6	29275	18828

2. **Pengaruh Kedalaman terhadap Jumlah Voxel dan Waktu**

Semakin besar depth octree, semakin banyak voxel boundary yang dihasilkan. Hal ini terjadi karena setiap node dapat dibagi lagi menjadi 8 child, sehingga representasi objek menjadi semakin detail. Namun, karena program hanya melanjutkan subdivisi pada node yang masih beririsan dengan permukaan objek, jumlah voxel yang terbentuk tidak bertambah sebesar pembagian penuh.

Dari hasil pengujian, semua model menunjukkan pola yang sama: depth yang lebih tinggi menghasilkan jumlah voxel yang lebih banyak dan waktu eksekusi yang lebih lama. Sebagai contoh, pada **`cow.obj`**, jumlah voxel naik dari 88 pada depth 3 menjadi 5621 pada depth 6. Waktu eksekusinya juga naik dari 51,43 ms menjadi 3567,21 ms. Ini menunjukkan bahwa semakin dalam subdivisi dilakukan, semakin besar pekerjaan yang harus diproses program.

3. **Perbandingan Antar Model**

Model yang paling ringan adalah **`line.obj`**. Hal ini wajar karena model ini hanya memiliki 1 face, sehingga pengecekan overlap sangat cepat. Pada depth 6, model ini hanya menghasilkan 1006 voxel dengan waktu eksekusi 1,36 ms.

Selanjutnya, model yang terberat adalah **`bowl.obj`** dan **`pumpkin.obj`**. Keduanya menghasilkan jumlah voxel yang besar pada depth 6, yaitu masing-masing 18828 dan 18635 voxel. Waktu eksekusinya juga paling tinggi, yaitu 29275,4 ms untuk **`bowl.obj`** dan 22827,8 ms untuk **`pumpkin.obj`**. Bisa dilihat bahwa bentuk dari pumpkin lebih kompleks, tetapi bowl memakan waktu lebih lama. Maka waktu eksekusi tidak hanya dipengaruhi oleh jumlah face atau kompleksitas bentuk secara visual, tetapi juga oleh bagaimana permukaan model tersebar di dalam ruang octree. Jika suatu model menyebabkan lebih banyak node tetap berstatus **BOUNDARY** pada banyak level, maka lebih

banyak cabang rekursi yang terus hidup sampai depth besar. Akibatnya, traversal tree dan jumlah overlap checking yang dilakukan juga menjadi lebih banyak.

Objek `cow.obj` dan `teapot.obj` berada di tengah. Objek `cow.obj` menghasilkan 5621 voxel pada depth 6 dengan waktu 3567,21 ms, sedangkan `teapot.obj` menghasilkan 7303 voxel dengan waktu 505,59 ms. Walaupun voxel `teapot.obj` lebih banyak, waktunya tetap jauh lebih cepat. Ini menunjukkan bahwa jumlah voxel akhir juga bukan satu-satunya faktor; jumlah face input dan pola penyebaran branch aktif di octree ikut memengaruhi waktu

H. Kesimpulan

Dari hasil implementasi dan pengujian, dapat disimpulkan bahwa algoritma divide and conquer dengan struktur data octree cukup efektif untuk melakukan boundary voxelization. Dengan membagi ruang secara rekursif dan menghentikan cabang yang kosong lebih awal, program tidak perlu memeriksa seluruh ruang secara brute force, menyebabkan program berjalan dengan komputasi yang lebih murah secara signifikan.

Penggunaan dua tahap pengecekan, yaitu AABB check dan triangle-box intersection check, juga membantu meningkatkan efisiensi karena pemeriksaan yang lebih mahal hanya dilakukan pada kandidat yang memiliki potensi tinggi untuk beririsan.

Selain itu, kedalaman octree sangat memengaruhi hasil voxelisasi. Semakin besar depth, semakin detail hasil yang diperoleh, tetapi waktu eksekusi juga meningkat. Performa program juga dipengaruhi oleh bentuk dan penyebaran permukaan objek, karena hal tersebut menentukan seberapa banyak cabang octree yang tetap aktif hingga kedalaman tinggi (semakin awal suatu voxel cube state **EMPTY**, semakin ringan komputasinya).

I. Saran

Dalam implementasi dasar ini, setiap node octree yang diperiksa masih mengiterasi seluruh face pada mesh untuk menentukan apakah cube tersebut beririsan dengan permukaan objek. Akibatnya, biaya pemeriksaan pada setiap node tetap bergantung pada jumlah total segitiga, meskipun cube yang diperiksa sebenarnya hanya mewakili bagian kecil dari ruang.

Keuntungan penggunaan octree tetap terasa karena jumlah node yang benar-benar diproses menjadi lebih sedikit; node yang kosong dapat dihentikan lebih awal sehingga tidak perlu dibagi lebih lanjut. Namun, efisiensi ini masih bisa ditingkatkan. Salah satu optimasi yang penting adalah **melokalisasi segitiga** pada setiap subruang, yaitu dengan menyimpan hanya subset segitiga yang relevan untuk masing-masing node atau voxel cube. Dengan begitu, saat memeriksa sebuah cube, program tidak perlu lagi membandingkannya dengan seluruh segitiga pada mesh, melainkan cukup dengan segitiga-segitiga yang memang berada di sekitar cube tersebut. Pendekatan ini dapat mengurangi jumlah pemeriksaan overlap secara signifikan dan membuat proses voxelisasi menjadi lebih efisien, dan juga sejalur dengan menggunakan tema strategi *divide and conquer*.

LAMPIRAN

A. Repository Program

Link Github:
https://github.com/tmthyberd/Tucil2_13524092

B. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Timothy Bernard Soeharto

C. Checklist Peniliaian

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✓	
2	Program berhasil dijalankan	✓	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✓	
4	Program informasi-informasi atau report	✓	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✓	
6	[Bonus] Program menggunakan *concurrency*		✓
7	[Bonus] Program dapat menampilkan visualisasi .obj		✓